# Methods in Computational Sciences

Andreane Maure, UNI : am6823

3/23/2026

## Question 1

We want to discretize the nonlinear reaction–diffusion equation

$$-\nabla^2 u + \gamma u^p = f$$

on the unit square, with Dirichlet boundary conditions given by the exact solution :

$$u = u_{\text{exact}} \qquad \text{on } \partial\Omega.$$

I use a uniform grid with spacing

$$h = \frac{1}{N-1},$$

and grid points

$$x_i = ih, \qquad y_j = jh, \qquad i, j = 0, \ldots, N-1.$$

Let $u_{i,j}$ denote the approximation of $u(x_i, y_j)$.
For an interior grid point, I approximate the Laplacian with the standard 5-point finite-difference stencil :

$$-\nabla^2 u(x_i, y_j) \approx \frac{4u_{i,j} - u_{i+1,j} - u_{i-1,j} - u_{i,j+1} - u_{i,j-1}}{h^2}.$$

So at each interior point, the discrete nonlinear equation becomes

$$\frac{4u_{i,j} - u_{i+1,j} - u_{i-1,j} - u_{i,j+1} - u_{i,j-1}}{h^2} + \gamma(u_{i,j})^p - f(x_i, y_j) = 0.$$

This gives the nonlinear residual

$$F_{i,j}(u) = \frac{4u_{i,j} - u_{i+1,j} - u_{i-1,j} - u_{i,j+1} - u_{i,j-1}}{h^2} + \gamma(u_{i,j})^p - f_{i,j},$$

for all interior points, where

$$f_{i,j} = f(x_i, y_j).$$

On the boundary, I impose the non-homogeneous Dirichlet condition directly. So for any boundary node,

$$F_{i,j}(u) = u_{i,j} - u_{\text{exact}}(x_i, y_j) = 0.$$

Therefore, the full nonlinear system can be written as

$$F(u) = 0,$$

with

$$F_{i,j}(u) = \begin{cases} \dfrac{4u_{i,j} - u_{i+1,j} - u_{i-1,j} - u_{i,j+1} - u_{i,j-1}}{h^2} + \gamma(u_{i,j})^p - f_{i,j}, & \text{for interior nodes,} \\ u_{i,j} - u_{\text{exact}}(x_i, y_j), & \text{for boundary nodes.} \end{cases}$$

## Question 2

Next, I derive the Jacobian matrix associated with this nonlinear system. The Jacobian is defined by

$$J(u) = \left[\frac{\partial F_\alpha}{\partial u_\beta}\right].$$

For an interior node $(i, j)$, the residual depends only on the value at the node itself and its four nearest neighbors. Starting from

$$F_{i,j}(u) = \frac{4u_{i,j} - u_{i+1,j} - u_{i-1,j} - u_{i,j+1} - u_{i,j-1}}{h^2} + \gamma(u_{i,j})^p - f_{i,j},$$

the derivative with respect to the central unknown $u_{i,j}$ is

$$\frac{\partial F_{i,j}}{\partial u_{i,j}} = \frac{4}{h^2} + \gamma p (u_{i,j})^{p-1}.$$

The derivatives with respect to the four neighboring points are

$$\frac{\partial F_{i,j}}{\partial u_{i+1,j}} = \frac{\partial F_{i,j}}{\partial u_{i-1,j}} = \frac{\partial F_{i,j}}{\partial u_{i,j+1}} = \frac{\partial F_{i,j}}{\partial u_{i,j-1}} = -\frac{1}{h^2}.$$

All other entries in that row are zero.
For a boundary node, the residual is simply

$$F_{i,j}(u) = u_{i,j} - u_{\text{exact}}(x_i, y_j),$$

so the Jacobian row is very simple :

$$\frac{\partial F_{i,j}}{\partial u_{i,j}} = 1,$$

and

$$\frac{\partial F_{i,j}}{\partial u_{k,l}} = 0 \qquad \text{for } (k,l) \neq (i,j).$$

So overall, for interior points the Jacobian has the usual 5-point stencil structure, with the nonlinear term adding

$$\gamma p (u_{i,j})^{p-1}$$

to the diagonal entry. On the boundary, the rows are replaced by identity rows in order to enforce the Dirichlet boundary conditions.

# Question 3

We modified the code `reaction.c` from the `p4pdes` repository into `reaction2d.c`, extending the 1D nonlinear solver to a 2D reaction-diffusion problem using PETSc SNES objects. The code is included in the submission.

# Question 4

We run the nonlinear solver with $\gamma = 0$ on a $65 \times 65$ grid and compare the result with the linear solver from `poisson2d.c`.

## Linear solver options

To achieve convergence in a single Newton iteration, we use a direct LU solver for the linear system at each Newton step. The options are saved in `options_file_gamma0` :

```
-rct_gamma 0.0
-rct_p 2
-da_grid_x 65
-da_grid_y 65
-ksp_type preonly
-pc_type lu
-snes_atol 1e-10
-snes_rtol 1e-10
```

The key insight is that with $\gamma = 0$, the problem is linear, so Newton's method should converge in exactly one iteration provided the linear system is solved accurately enough. Using `-ksp_type preonly -pc_type lu` replaces the iterative Krylov solver with a direct LU factorization, which solves the system to machine precision in a single step.

## Results

Running with the above options gives :

```
  0 SNES Function norm 8.867115559165e+00
  1 SNES Function norm 1.930702051844e-11
on 65 x 65 grid:  rel_error |u-uexact|_2/|uexact|_2 = 0.000524515
```

The nonlinear solver converges in **1 Newton iteration**, with a final residual norm $\|F(u)\|_2 = 1.93 \times 10^{-11} < 10^{-10}$ as required. At each Newton step, 1 KSP iteration is taken (the direct LU solve).

### Comparison with the linear solver

Running `poisson2d` on the same $65 \times 65$ grid gives :

```
on 65 x 65 grid:  rel_error |u-uexact|_2/|uexact|_2 = 0.000515825
```

The two solvers produce the same solution within the discretization error :

| Solver | $\|u - u_{\text{exact}}\|_2 / \|u_{\text{exact}}\|_2$ | Newton iterations |
|---|---|---|
| `poisson2d` (linear, KSP) | $5.158 \times 10^{-4}$ | – |
| `reaction2d` $\gamma = 0$ (SNES) | $5.245 \times 10^{-4}$ | 1 |

TABLE 1 – Comparison of linear and nonlinear solvers on a $65 \times 65$ grid with $\gamma = 0$.

The relative difference between the two errors is less than 2%, which is consistent with the fact that the two codes use slightly different formulations of the discrete residual (integrated form vs. strong form), but both converge to the same continuous solution as the grid is refined.

## Question 5

We verify that the finite-difference Jacobian produces the same solution as the user-provided analytical Jacobian, and compare their run times using `-log_view`.

### Results

Both runs use the same options as in Question 4 (`options_file_gamma0`), with `-snes_fd` added for the finite-difference case.

| Jacobian | Newton iterations | $\|F(u)\|_2$ final | Relative error | `SNESSolve` time |
|---|---|---|---|---|
| Analytical | 1 | $1.93 \times 10^{-11}$ | $5.245 \times 10^{-4}$ | 0.048s |
| Finite difference | 2 | $1.92 \times 10^{-11}$ | $5.245 \times 10^{-4}$ | 2.68s |

TABLE 2 – Comparison of analytical and finite-difference Jacobians on a $65 \times 65$ grid with $\gamma = 0$.

### Discussion

Both Jacobians converge to the same final solution, with identical relative error $\|u - u_{\text{exact}}\|_2 / \|u_{\text{exact}}\|_2 = 5.245 \times 10^{-4}$.
However, there are notable differences in convergence behavior and run time :

- **Newton iterations :** The analytical Jacobian converges in 1 Newton iteration, while the finite-difference Jacobian requires 2. This is because the finite-difference Jacobian is only an approximation of the true Jacobian (accurate to $O(h)$), introducing a small error that prevents exact quadratic convergence.
- **Run time :** The finite-difference Jacobian is approximately 55× slower (2.68s vs. 0.048s). This is expected : `-snes_fd` computes the Jacobian by perturbing each degree of freedom independently, requiring one evaluation of $F(u)$ per column of the matrix. On a $65 \times 65$ grid there are $65^2 = 4225$ unknowns, leading to 8455 function evaluations (`SNESFunctionEval`) compared to just 2 with the analytical Jacobian.

The options used for the finite-difference run are saved in `options_file_fd` :

```
-rct_gamma 0.0
-rct_p 2
-da_grid_x 65
-da_grid_y 65
-ksp_type preonly
-pc_type lu
-snes_atol 1e-10
-snes_rtol 1e-10
-snes_fd
```

# Question 6

We run the nonlinear solver with $\gamma = 100$ and $p = 3$ on a $65 \times 65$ grid, using the same stopping conditions as in Question 4 (`-snes_atol 1e-10 -snes_rtol 1e-10`, direct LU solver).

## Results

```
  0 SNES Function norm 8.867115559165e-01
    0 KSP Residual norm 8.867115559165e-01
    1 KSP Residual norm 1.614443832572e-14
  1 SNES Function norm 7.777424323105e-05
    0 KSP Residual norm 7.777424323105e-05
    1 KSP Residual norm 1.828830200672e-18
  2 SNES Function norm 1.905055223210e-11
on 65 x 65 grid:  rel_error |u-uexact|_2/|uexact|_2 = 0.000202508
```

| Newton step | $\|F(u)\|_2$ | KSP iterations |
|---|---|---|
| 0 | $8.867 \times 10^{-1}$ | – |
| 1 | $7.777 \times 10^{-5}$ | 1 |
| 2 | $1.905 \times 10^{-11}$ | 1 |

TABLE 3 – Newton convergence for $\gamma = 100$, $p = 3$ on a $65 \times 65$ grid.

The solver converges in **2 Newton iterations**, each requiring a single KSP iteration (direct LU solve). The final residual norm $\|F(u)\|_2 = 1.91 \times 10^{-11} < 10^{-10}$ satisfies the required tolerance. The relative error $\|u - u_{\text{exact}}\|_2 / \|u_{\text{exact}}\|_2 = 2.025 \times 10^{-4}$ is smaller than in the $\gamma = 0$ case ($5.245 \times 10^{-4}$), which is expected since the reaction term stiffens the problem and concentrates the solution.

## Convergence history

Figure 1 shows the convergence history of the nonlinear residual norm. The convergence is clearly quadratic : the residual drops from $\sim 10^0$ to $\sim 10^{-5}$ in one iteration, then to $\sim 10^{-11}$ in the next, consistent with Newton's method applied to a smooth nonlinear problem.

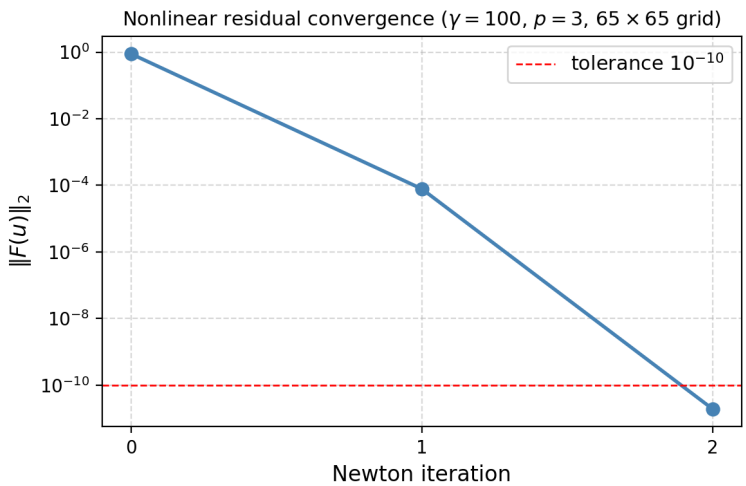



FIGURE 1 – Convergence history of $\|F(u)\|_2$ for $\gamma = 100$, $p = 3$, $65 \times 65$ grid. The dashed red line indicates the tolerance $10^{-10}$.

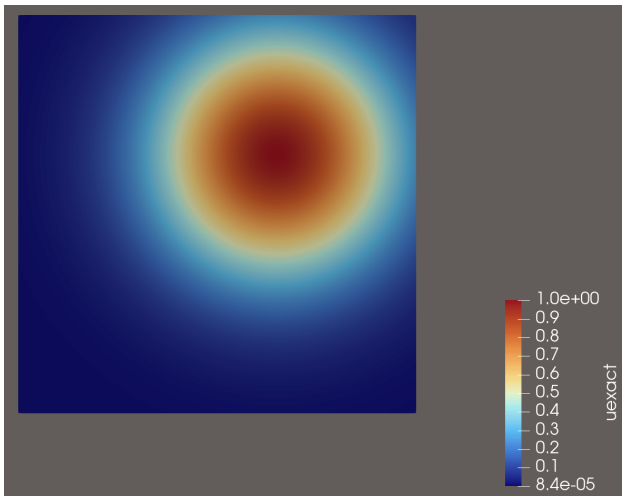
# Question 7

We rerun the problem with `-rct_linear_f` for $\gamma = 100$ and $p = 3$ on a $65 \times 65$ grid :

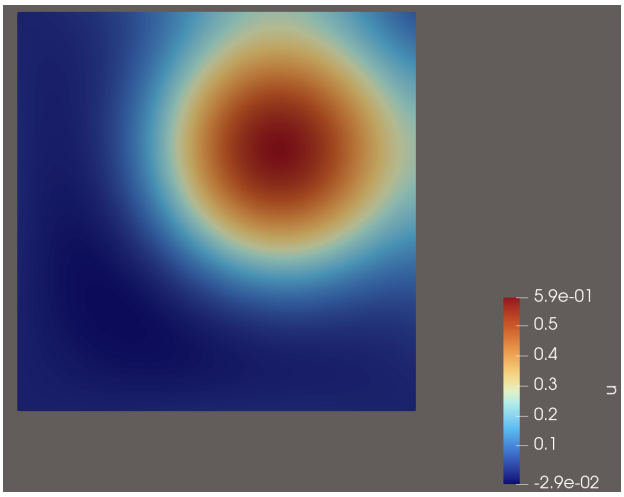
```
mpirun -n 1 ./reaction2d -rct_gamma 100.0 -rct_p 3 -da_grid_x 65 -da_grid_y 65
    -ksp_type preonly -pc_type lu -snes_atol 1e-10 -snes_rtol 1e-10
    -rct_linear_f -snes_monitor

  0 SNES Function norm 1.388554447210e+03
  1 SNES Function norm 3.019323015697e+02
  2 SNES Function norm 3.160730762479e+01
  3 SNES Function norm 5.060906539575e-01
  4 SNES Function norm 1.425197042867e-04
  5 SNES Function norm 1.775579972919e-11
on 65 x 65 grid:  rel_error |u-uexact|_2/|uexact|_2 = 0.407779
```

## ParaView visualization




(a) Exact solution $u_{\text{exact}}$ : perfect Gaussian bump centered at $(0.65, 0.65)$, maximum amplitude 1.0.




(b) Numerical solution $u$ with `-rct_linear_f` : distorted Gaussian, maximum $\sim 0.59$, negative values near boundary.

FIGURE 2 – Comparison of $u_{\text{exact}}$ and the numerical solution $u$ for $\gamma = 100$, $p = 3$, $65 \times 65$ grid, with `-rct_linear_f`.

## Discussion

With `-rct_linear_f`, the right-hand side is set to $f = -\nabla^2 u_{\text{exact}}$ only, omitting the nonlinear manufactured term $\gamma u_{\text{exact}}^p$. This means we are solving the *wrong* problem : the nonlinear operator $-\nabla^2 u + \gamma u^p$ is applied to the solution, but the RHS does not account for the reaction term. As a result :

- The solver requires **6 Newton iterations** to converge (vs. 2 in Question 6), reflecting the much larger initial residual ($\|F(u_0)\|_2 \approx 1.4 \times 10^3$).
- The relative error is $\|u - u_{\text{exact}}\|_2 / \|u_{\text{exact}}\|_2 = 0.408$, roughly 2000 times larger than in the correct nonlinear case ($2.025 \times 10^{-4}$). The solver has converged, but to the wrong solution.
- As visible in Figure 2, the numerical solution is a distorted Gaussian : the maximum amplitude is reduced to $\sim 0.59$ (vs. 1.0 for $u_{\text{exact}}$), and the solution takes small negative values ($\sim -0.03$) near the boundary — both physically inconsistent with the exact Gaussian solution.

This illustrates the importance of using the correct nonlinear RHS when applying the method of manufactured solutions : omitting $\gamma u_{\text{exact}}^p$ from $f$ introduces a large error that completely dominates the discretization error, especially for large $\gamma$.

# Question 8

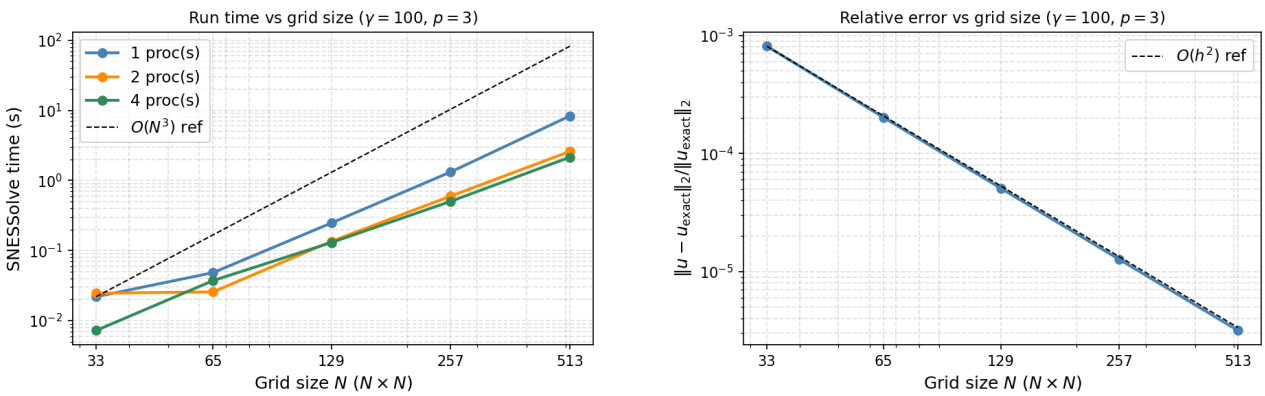
We study the performance and scaling of the nonlinear solver for $\gamma = 100$, $p = 3$ with `-ksp_type preonly -pc_type lu` across grid sizes from $33 \times 33$ to $513 \times 513$ and 1, 2, and 4 processors.

## Results

All runs converge in **2 Newton iterations** regardless of grid size or number of processors, confirming the quadratic convergence of Newton's method.

| Grid | Procs | Time (s) | Rel. error | Newton iter |
|---|---|---|---|---|
| $33 \times 33$ | 1 | 0.0219 | $8.10 \times 10^{-4}$ | 2 |
| $33 \times 33$ | 2 | 0.0246 | $8.10 \times 10^{-4}$ | 2 |
| $33 \times 33$ | 4 | 0.0072 | $8.10 \times 10^{-4}$ | 2 |
| $65 \times 65$ | 1 | 0.0484 | $2.03 \times 10^{-4}$ | 2 |
| $65 \times 65$ | 2 | 0.0257 | $2.03 \times 10^{-4}$ | 2 |
| $65 \times 65$ | 4 | 0.0373 | $2.03 \times 10^{-4}$ | 2 |
| $129 \times 129$ | 1 | 0.2477 | $5.06 \times 10^{-5}$ | 2 |
| $129 \times 129$ | 2 | 0.1354 | $5.06 \times 10^{-5}$ | 2 |
| $129 \times 129$ | 4 | 0.1302 | $5.06 \times 10^{-5}$ | 2 |
| $257 \times 257$ | 1 | 1.3194 | $1.27 \times 10^{-5}$ | 2 |
| $257 \times 257$ | 2 | 0.5967 | $1.27 \times 10^{-5}$ | 2 |
| $257 \times 257$ | 4 | 0.5022 | $1.27 \times 10^{-5}$ | 2 |
| $513 \times 513$ | 1 | 8.3289 | $3.17 \times 10^{-6}$ | 2 |
| $513 \times 513$ | 2 | 2.5998 | $3.17 \times 10^{-6}$ | 2 |
| $513 \times 513$ | 4 | 2.1458 | $3.17 \times 10^{-6}$ | 2 |

TABLE 4 – Performance and scaling results for $\gamma = 100$, $p = 3$.




(a) Run time vs grid size for 1, 2, and 4 processors.

(b) Relative error vs grid size, showing $O(h^2)$ convergence.

FIGURE 3 – Scaling results for $\gamma = 100$, $p = 3$.

## Discussion

**Error convergence :** The relative error decreases as $O(h^2)$ with grid refinement, as expected from the second-order finite difference discretization. Each refinement level (doubling $N$) reduces the error by a factor of $\sim 4$.
**Run time scaling with problem size :** The run time grows roughly as $O(N^3)$ with the grid size $N$. This is expected for a direct LU solver on a 2D problem with $N^2$ unknowns : the LU factorization of a sparse banded matrix of size $N^2 \times N^2$ costs $O(N^3)$ in 2D.
**Parallel scaling :** The parallel scaling is modest on small grids (where communication overhead dominates) but improves for larger problems. Going from 1 to 2 processors reduces the time by roughly $3\times$ for the $513 \times 513$ grid (8.33s $\rightarrow$ 2.60s), and going from 1 to 4 processors gives roughly $4\times$ speedup (8.33s $\rightarrow$ 2.15s). This near-linear speedup on the largest grid suggests good parallel efficiency for large problems. However, the direct LU solver (`-pc_type lu`) is inherently sequential in its factorization phase, which limits parallel scalability compared to iterative solvers with parallel preconditioners.